

HUONG DAN KY THUAT

Xay dung Self-Hosted Search Engine & Dong goi APK

Huong dan chi tiet tu dau den cuoc: tu web crawler, inverted index, thuat toan ranking cho den viec dong goi ung dung tim kiem thanh file APK Android - hoan toan doc lap, khong can API ben thu 3.

Z.ai Technical Guide

Thang 5, 2026

Muc luc

Chuong 1: Gioi thieu ve Cong cu Tim kiem Tu xay dung	3
1.1. Tai sao nen tu xay dung Search Engine?	3
Chuong 2: Kien truc Tong quan cua He thong Search	4
2.1. Cac thanh phan chinh	4
2.2. Quy trinh hoat dong	4
Chuong 3: Cong nghe va Cong cu Khuyen nghi	5
3.1. Ngon ngu lap trinh	5
3.2. He quan tri co so du lieu	5
3.3. Cong cu dong goi APK	6
Chuong 4: Xay dung Web Crawler (Spider)	6
4.1. Nguyen ly hoat dong	6
4.2. Vi du code Web Crawler don gian voi Python	7
4.3. Cac luu y quan trong khi xay dung Crawler	8
Chuong 5: He thong Indexing - Inverted Index	9
5.1. Inverted Index la gi?	9
5.2. Xay dung Inverted Index voi Whoosh	9
5.3. Tim kiem voi Whoosh	10
Chuong 6: Thuat toan Search va Ranking	11
6.1. TF-IDF (Term Frequency - Inverse Document Frequency)	11
6.2. BM25 (Okapi BM25)	11
6.3. So sanh cac thuat toan	12
Chuong 7: Xay dung Giao dien Web Frontend	12
7.1. Backend API voi Flask	12

7.2. Frontend đơn giản với HTML + JavaScript	13
7.3. Tùy chọn Framework frontend hiện đại	14
Chương 8: Đóng gói APK Android (WebView)	14
8.1. Phương pháp 1: WebView Activity (Android Studio)	14
8.2. Phương pháp 2: Apache Cordova / Capacitor	15
8.3. Phương pháp 3: PWA (Progressive Web App)	16
Chương 9: Triển khai và Bảo trì Hệ thống	16
9.1. Triển khai Backend Server	16
9.2. Triển khai Offline (cho APK)	17
9.3. Giám sát và bảo trì	17
Chương 10: Tổng kết và Lời khuyên	17
10.1. Lời khuyên cho người mới bắt đầu	18
10.2. Chi phí ước tính	18

Chương 1: Giới thiệu về Công cụ Tìm kiếm Tự xây dựng

Trong thời đại số hiện nay, việc tìm kiếm thông tin trên Internet là nhu cầu cơ bản của mọi người. Các công cụ tìm kiếm như Google, Bing, DuckDuckGo đã trở nên quen thuộc, nhưng hoàn toàn phụ thuộc vào server của bên thứ ba. Điều này gây ra nhiều vấn đề về bảo mật, quyền riêng tư và tốc độ truy cập. Ngoài ra, bạn cũng mất phí khi sử dụng API của bên thứ ba cho các dự án riêng, đặc biệt là khi mức truy cập tăng cao.

Vì vậy, việc tự xây dựng một hệ thống tìm kiếm riêng tư (self-hosted search engine) là giải pháp tối ưu. Hệ thống này cho phép bạn lưu trữ, chỉ mục và tìm kiếm dữ liệu trên server của bạn hoặc thiết bị cục bộ, hoàn toàn độc lập với các dịch vụ bên ngoài. Bạn có thể áp dụng mô hình này cho nhiều trường hợp sử dụng: tìm kiếm nội bộ trong công ty, tìm kiếm tài liệu riêng, hoặc xây dựng ứng dụng APK cho điện thoại Android với khả năng tìm kiếm offline.

Tài liệu này sẽ hướng dẫn bạn từ đầu đến cuối, từ việc thiết kế kiến trúc, lựa chọn công nghệ, xây dựng web crawler, tạo hệ thống chỉ mục (inverted index), triển khai thuật toán ranking, đến việc đóng gói thành ứng dụng APK Android. Mọi bước đều được giải thích chi tiết kèm theo ví dụ code thực tế để bạn dễ dàng theo dõi và áp dụng.

1.1. Tại sao nên tự xây dựng Search Engine?

Sử dụng dịch vụ tìm kiếm của bên thứ ba (Google Custom Search API, Bing Search API) tuy tiện nhưng có nhiều hạn chế. Thứ nhất, bạn phụ thuộc hoàn toàn vào bên cung cấp: nếu họ thay đổi chính sách giá hoặc ngừng dịch vụ, hệ thống của bạn sẽ bị ảnh hưởng trực tiếp. Thứ hai, các API này có giới hạn số lượng request (thường từ 100 đến 1000 query/ngày cho miền free), không phù hợp với các ứng dụng có lượng người dùng lớn. Thứ ba, bạn không thể kiểm soát được chất lượng kết quả và thời gian phản hồi vì server nằm ở xa.

Ngoài ra, một hệ thống tìm kiếm tự host mang lại nhiều ưu đãi vượt trội. Bạn sở hữu trọn vẹn dữ liệu và có thể tùy chỉnh mọi khía cạnh của hệ thống theo nhu cầu cụ thể. Tốc độ xử lý nhanh hơn vì server nằm gần người dùng, và bạn không bị hạn chế về số lượng query. Đặc biệt, hệ thống có thể hoạt động ngay cả khi không có kết nối Internet (offline mode), rất phù hợp cho các ứng dụng di động được đóng gói thành file APK.

Chương 2: Kiến trúc Tổng quan của Hệ thống Search

Một hệ thống tìm kiếm hoàn chỉnh bao gồm nhiều thành phần chính làm việc kết hợp với nhau. Hiểu rõ kiến trúc tổng quan giúp bạn thiết kế và triển khai hiệu quả hơn. Dưới đây là các thành phần cốt lõi và cách chúng tương tác với nhau trong toàn bộ quy trình tìm kiếm.

2.1. Các thành phần chính

Thành phần	Chức năng	Công nghệ gợi ý
Web Crawler (Spider)	Thu thập dữ liệu từ các trang web, tải nội dung HTML, lưu vào cơ sở dữ liệu	Scrapy (Python), BeautifulSoup, requests
Indexer	Phân tích nội dung, tạo chỉ mục đảo (inverted index) để tìm kiếm nhanh	Whoosh, Elasticsearch, Meilisearch
Query Engine	Nhận query từ người dùng, phân tích và chuyển đổi thành câu lệnh tìm kiếm	Python, Node.js, Go
Ranking Engine	Xếp hạng kết quả dựa trên điểm phù hợp (relevance score)	TF-IDF, BM25, PageRank
Frontend / API	Cung cấp giao diện người dùng hoặc API endpoint để truy vấn	Next.js, Flask, WebView (Android)
Database / Storage	Lưu trữ dữ liệu gốc, chỉ mục và metadata	SQLite, PostgreSQL, Redis

Bảng 1: Các thành phần chính của hệ thống tìm kiếm tự xây dựng

2.2. Quy trình hoạt động

Quy trình hoạt động của một hệ thống tìm kiếm tự host được chia thành hai giai đoạn chính: giai đoạn xây dựng chỉ mục (indexing phase) và giai đoạn truy vấn (query phase). Trong giai đoạn đầu tiên, Web Crawler sẽ đi lan trên các trang web mà bạn đã cấu hình, tải nội dung về và lưu vào cơ sở dữ liệu. Sau đó, Indexer sẽ phân tích nội dung, loại bỏ tag HTML, tách từ khóa và tạo ra cấu trúc dữ liệu chỉ mục đảo (inverted index) cho phép tìm kiếm cực kỳ nhanh.

Trong giai đoạn truy vấn, khi người dùng nhập một từ khóa vào ô tìm kiếm, Query Engine sẽ nhận yêu cầu, phân tích từ khóa (tokenize), gợi ý từ gợi ý (autocorrect, synonym), rồi truy vấn vào inverted index để lấy ra danh sách các

tai lieu chua tu khoa do. Ranking Engine se tinh toan diem phu hop cho tung tailieu dua tren cac thuat toan nhu BM25 hoac TF-IDF, sau do sap xep ket qua vatra ve cho nguoi dung danh sach da duoc xep hang.

Chương 3: Công nghệ và Công cụ Khuyến nghị

Việc lựa chọn công nghệ là bước quan trọng nhất khi bắt đầu xây dựng hệ thống tìm kiếm. Mỗi công nghệ có ưu nhược điểm riêng, và việc kết hợp chúng một cách hợp lý sẽ tạo ra hệ thống hoạt động ổn định, hiệu quả và dễ nâng cấp. Dưới đây là danh sách các công nghệ khuyến nghị cho mọi thành phần của hệ thống.

3.1. Ngôn ngữ lập trình

Python là lựa chọn tốt nhất cho việc xây dựng hệ thống tìm kiếm từ host, bởi vì nó có sinh thái thư viện phong phú và cài đặt đơn giản. Cho web crawler, bạn có thể sử dụng Scrapy (hiệu năng cao, hỗ trợ phân tán) hoặc BeautifulSoup (đơn giản, dễ sử dụng). Cho hệ thống indexing, Whoosh là thư viện Python pure (không cần phụ thuộc bên ngoài), phù hợp cho các hệ thống nhỏ và vừa. Meilisearch là một lựa chọn khác tuyệt vời với khả năng tìm kiếm full-text nhanh chóng và hỗ trợ typo tolerance.

Nếu bạn muốn xây dựng frontend hiện đại, Next.js (React) hoặc Vue.js là những lựa chọn tốt. Cho backend API, Flask hoặc FastAPI (Python) là những framework nhẹ, nhanh và dễ triển khai. Nếu hiệu năng là ưu tiên tiên, Go (Golang) với thư viện bleve hoặc Zinc là những lựa chọn xuất sắc.

3.2. Hệ quản trị cơ sở dữ liệu

He DB	Loai	Uu diem	Phu hop cho
SQLite	File-based, Embeddable	Không cần cài đặt server, lưu 1 file, tốt cho mobile	APK offline, prototype, dữ liệu nhỏ
PostgreSQL	RDBMS	Mạnh mẽ, hỗ trợ full-text search, toàn diện	Hệ thống production, dữ liệu lớn
Redis	In-memory Cache	Tốc độ cực nhanh, hỗ trợ caching kết quả	Cache query, session, ranking

Whoosh	Inverted Index (Python)	Pure Python, de cai dat, khong can server	Tim kiem van ban, du lieu vua
Meilisearch	Search Engine	Full-text search nhanh, typo tolerance, easy API	Ung dung can tim kiem thong minh

Bang 2: So sanh cac he quan tri co so du lieu pho bien

3.3. Cong cu dong goi APK

De bien website tim kiem thanh ung dung APK Android, ban co nhieu cach khac nhau. Cach don gian nhat la su dung WebView de dong goi trang web thanh ung dung Android. Ban co the dung Android Studio de tao mot project moi voi WebView Activity, hoac su dung cac cong cu nhu Apache Cordova, Capacitor (tu Ionic), hoac PWA Builder de chuyen doi trang web thanh PWA (Progressive Web App) co the cai dat nhu APK.

Dau vao la he thong tim kiem hien co (website hoac backend API), dau ra la file APK ma nguoi dung co the tai ve va cai dat tren dien thoai Android. Ung dung nay se su dung WebView de hien thi giao dien tim kiem va giao tiep voi backend thong qua API endpoint. Neu ban muon ung dung hoat dong hoan toan offline, ban can dong goi ca database (SQLite) chua chi muc tim kiem cung voi APK.

Chuong 4: Xay dung Web Crawler (Spider)

Web Crawler (hay Spider) la thanh phan dau tien va quan trong nhat cua he thong tim kiem. No chiu trach nhiem thu thap du lieu tu cac nguon (website, file, API), xu ly noi dung va luu tru vao co so du lieu de sau do Indexer co the tao chi muc. Vi tri chat luong cua crawler quyet dinh chat luong cua toan bo he thong tim kiem.

4.1. Nguyen ly hoat dong

Web Crawler hoat dong theo quy trinh nhu sau: Ban dau, ban cung cap danh sach cac URL seed (URL ban dau). Crawler se gui yeu cau HTTP den tung URL, tai noi dung HTML ve, phan tich va rut trich thong tin can thiet (tieu de, mo ta, noi dung chinh, link). Sau do, no tim tat ca cac link moi trong trang, kiem tra xem da thu thap chua, va them vao hang doi (queue) de thu thap tiep. Quy trinh

nay lap lai cho den khi hang doi trong hoac dat duoc gioi han so luong trang dacao hinh.

Mot crawler hieu qua can co cac tinh nang: tuan thu robots.txt, gioi han toc do request de khong qua tai server muc tieu (rate limiting), kha nang xu ly loi (retry khi fail), ho tro user-agent rotation de tranh bi chan, va kha nang luu trang thai thu thap de tiep tuc sau khi dung (resume capability).

4.2. Vi du code Web Crawler don gian voi Python

Duoi day la vi du ve mot web crawler don gian su dung requests va BeautifulSoup de thu thap noi dung tu mot trang web. Crawler nay se tai noi dung, rut trich tieu de, mo ta va noi dung chinh, sau do luu vao database SQLite.

```
import requests
from bs4 import BeautifulSoup
import sqlite3
from urllib.parse import urljoin, urlparse
from collections import deque
import time

class SimpleCrawler:
    def __init__(self, seed_urls, db_path='crawler.db'):
        self.queue = deque(seed_urls)
        self.visited = set()
        self.conn = sqlite3.connect(db_path)
        self._init_db()

    def _init_db(self):
        cur = self.conn.cursor()
        cur.execute('''CREATE TABLE IF NOT EXISTS pages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            url TEXT UNIQUE,
            title TEXT,
            description TEXT,
            content TEXT,
            crawled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )''')
        self.conn.commit()

    def crawl(self, max_pages=100, delay=1.0):
        while self.queue and len(self.visited) < max_pages:
            url = self.queue.popleft()
            if url in self.visited:
                continue
            try:
                resp = requests.get(url, timeout=10,
                                    headers={'User-Agent': 'MySearchBot/1.0'})
                resp.raise_for_status()
```

```

soup = BeautifulSoup(resp.text, 'html.parser')

title = soup.title.string if soup.title else ''
desc_tag = soup.find('meta',
    attrs={'name': 'description'})
description = desc_tag['content'] if desc_tag else ''

# Remove script/style tags
for tag in soup(['script', 'style']):
    tag.decompose()
content = soup.get_text(strip=True)

# Save to database
self.conn.execute(
    'INSERT OR IGNORE INTO pages '
    '(url, title, description, content) '
    'VALUES (?, ?, ?, ?)',
    (url, title, description, content))
self.conn.commit()

# Extract new links
for a_tag in soup.find_all('a', href=True):
    new_url = urljoin(url, a_tag['href'])
    if urlparse(new_url).scheme in ('http', 'https'):
        if new_url not in self.visited:
            self.queue.append(new_url)

self.visited.add(url)
print(f'Crawled: {url}')
time.sleep(delay)

except Exception as e:
    print(f'Error crawling {url}: {e}')

```

4.3. Các lưu ý quan trọng khi xây dựng Crawler

Thứ nhất, bạn luôn phải kiểm tra file robots.txt của website mục tiêu trước khi thu thập. File này xác định các đường dẫn được phép hoặc bị cấm thu thập. Thứ hai, hãy đặt giới hạn tốc độ request (thường 1-2 giây giữa mỗi request) để tránh quá tải server mục tiêu và bị chặn IP. Thứ ba, hãy sử dụng User-Agent trung thực để nhận diện crawler của bạn. Thứ tư, cần có cơ chế xử lý lỗi: khi một request fail, hãy thử lại vài lần với delay tăng dần (exponential backoff) trước khi bỏ qua URL đó.

Thứ năm, hãy lưu trạng thái thu thập (danh sách đã visit, queue) vào file hoặc database để có thể tiếp tục crawler sau khi dừng. Thứ sáu, đối với các website lớn, hãy sử dụng proxy rotation để tránh bị chặn IP. Cuối cùng, hãy thiết

lap cau hinh domain filter (chi thu thap cac trang trong mot danh sach domaincho phep) de han che pham vi thu thap va tranh crawl vo han.

Chuong 5: He thong Indexing - Inverted Index

Sau khi thu thap du lieu, buoc tiep theo la tao chi muc (index) de phep tim kiem nhanh. Kieu chi muc pho biest nhat va hieu qua nhat cho tim kiem van ban la Inverted Index (chi muc dao). No la cau truc du lieu giup tim kiem mot tu khoa trong hang trieu tai lieu chi trong vai mili-giay.

5.1. Inverted Index la gi?

Inverted Index la mot cau truc du lieu anh xa tu tu khoa (term) den danh sach cac tai lieu chua tu khoa do. Nguoc voi "forward index" (anh xa tu tai lieu den danh sach tu), inverted index cho phep tim kiem cuc ky nhanh vi ban chi can tim tu khoa trong tu dien va lay ngay danh sach tai lien quan.

Vi du, neu ban co 3 tai lieu: Doc1 ("Python la ngon ngu lap trinh"), Doc2 ("Java la ngon ngu lap trinh"), Doc3 ("Python duoc su dung rong rai"), thi inverted index se la: "Python" - [Doc1, Doc3], "la" - [Doc1, Doc2], "ngon" - [Doc1, Doc2], "ngu" - [Doc1, Doc2], "lap" - [Doc1, Doc2], "trinh" - [Doc1, Doc2], "Java" - [Doc2], "duoc" - [Doc3], "su" - [Doc3], "dung" - [Doc3], "rong" - [Doc3], "rai" - [Doc3]. Khi nguoi dung tim "Python", he thong chi can tra cuu tu "Python" trong index va ngay lap tuc lay duoc ket qua la Doc1 va Doc3.

5.2. Xay dung Inverted Index voi Whoosh

Whoosh la thu vien Python pure ho tro tao va truy van inverted index mot cach don gian va hieu qua. Duoi day la vi du ve cach tao index tu du lieu da thu thap:

```
from whoosh.index import create_in, open_dir, exists_in
from whoosh.fields import Schema, TEXT, ID, STORED
from whoosh.analysis import StemmingAnalyzer
import os, sqlite3

# Define schema
schema = Schema(
    url=ID(stored=True, unique=True),
    title=TEXT(stored=True),
    content=TEXT(analyzer=StemmingAnalyzer(), stored=True),
    description=TEXT(stored=True)
)
```

```

# Create index directory
index_dir = 'search_index'
if not os.path.exists(index_dir):
    os.makedirs(index_dir)

if not exists_in(index_dir):
    ix = create_in(index_dir, schema)
else:
    ix = open_dir(index_dir)

# Build index from crawled data
writer = ix.writer()
conn = sqlite3.connect('crawler.db')
cur = conn.cursor()
cur.execute('SELECT url, title, description, content FROM pages')

for row in cur.fetchall():
    url, title, desc, content = row
    writer.update_document(
        url=url,
        title=title or '',
        content=content or '',
        description=desc or ''
    )
writer.commit()
print('Index created successfully!')

```

5.3. Tim kiem voi Whoosh

Sau khi index da duoc tao, ban co the thuc hien truy van tim kiem rat don gian. Whoosh ho tro nhieu kieu truy van: tim kiem co ban, tim kiem voi toan tu (AND, OR, NOT), tim kiem phu hop (fuzzy search), va tim kiem theo truong cu the. Duoi day la vi du ve cach thuc hien tim kiem co ban:

```

from whoosh.qparser import QueryParser, MultifieldParser

# Single field search
with ix.searcher() as searcher:
    query = QueryParser('content', ix.schema).parse('Python')
    results = searcher.search(query)
    for hit in results:
        print(f'Score: {hit.score:.2f}')
        print(f'Title: {hit["title"]}')
        print(f'URL: {hit["url"]}')

# Multi-field search (title + content)
with ix.searcher() as searcher:
    parser = MultifieldParser(

```

```
['title', 'content', 'description'], ix.schema)
query = parser.parse('lap trinh Python')
results = searcher.search(query, limit=20)
for hit in results:
    print(f'{hit["title"]} ({hit.score:.2f})')
```

Chương 6: Thuật toán Search và Ranking

Thuật toán ranking là trái tim của mọi hệ thống tìm kiếm. Nó quyết định kết quả nào sẽ hiển thị trước, kết quả nào phù hợp nhất với nhu cầu của người dùng. Không có thuật toán ranking tốt, một hệ thống có thể tìm ra hàng nghìn kết quả nhưng người dùng không thể tìm được điều họ cần.

6.1. TF-IDF (Term Frequency - Inverse Document Frequency)

TF-IDF là thuật toán ranking cơ bản nhất và được sử dụng rộng rãi. Nó đánh giá mức độ quan trọng của một từ trong một tài liệu dựa trên hai yếu tố. Thứ nhất là Tần suất Từ (TF): một từ xuất hiện càng nhiều trong tài liệu thì càng phù hợp. TF được tính bằng số lần xuất hiện của từ chia tổng số từ trong tài liệu. Thứ hai là Tần suất Tài liệu Đảo (IDF): một từ xuất hiện trong ít tài liệu thì càng có giá trị phân biệt cao. IDF được tính bằng $\log(\text{tổng số tài liệu} / \text{số tài liệu chứa từ})$.

Điểm TF-IDF = TF x IDF. Ví dụ, từ "Python" xuất hiện nhiều lần trong một tài liệu (TF cao) và chỉ xuất hiện trong một số ít tài liệu (IDF cao), thì điểm TF-IDF sẽ cao, cho thấy tài liệu đó phù hợp rất tốt với từ khóa "Python". Ngược lại, từ "la" xuất hiện nhiều nhưng có trong hầu hết tài liệu, nên IDF sẽ rất thấp, làm điểm TF-IDF thấp, cho thấy từ này không có giá trị phân biệt.

6.2. BM25 (Okapi BM25)

BM25 là phiên bản cải tiến của TF-IDF và là thuật toán ranking phổ biến nhất hiện nay, được sử dụng bởi nhiều hệ thống tìm kiếm lớn (Elasticsearch, Lucene, Meilisearch). BM25 giải quyết vấn đề của TF-IDF là khi một từ xuất hiện quá nhiều lần trong tài liệu, TF-IDF vẫn cho điểm cao, dù từ đó có thể không phù hợp hơn. BM25 áp dụng một hàm bão hòa (saturation function) để giới hạn tác động của TF, tương lai một từ xuất hiện quá nhiều sẽ không tăng điểm vô hạn.

Công thức BM25: $\text{Score}(D, Q) = \sum \text{over terms } q \text{ in } Q \text{ của } \text{IDF}(q_i) \times (f(q_i, D) \times (k_1 + 1)) / (f(q_i, D) + k_1 \times (1 - b + b \times |D| / \text{avgdl}))$. Trong đó, $f(q_i, D)$ là

tan suat tu qi trong tai lieu D, $|D|$ la do dai tai lieu D, $avgdl$ la do dai trung binh cua tat ca tai lieu, k_1 (thuong 1.2-2.0) la tham so dieu chinh muc do quan trong cua tan suat tu, va b (thuong 0.75) la tham so dieu chinh tac dong cua do dai tai lieu.

6.3. So sanh cac thuat toan

Thuat toan	Uu diem	Nhuoc diem	Truong hop su dung
TF-IDF	Don gian, de hieu, de trien khai	Khong gioi han TF, thieu context	Prototype, he thong nho
BM25	Chinh xac hon TF-IDF, ho tro tuning	Can tham so dieu chinh (k_1 , b)	He thong production, default choice
PageRank	Xep hang theo muc do uy tin trang	Chi danh cho web, can du lieu link	Tim kiem web, xep hang trang
Semantic Search	Hieu y nghĩa, ho tro tu dong nghĩa	Can model AI/ML, nang cao	Tim kiem thong minh, chatbot

Bang 3: So sanh cac thuat toan ranking pho bien

Chương 7: Xây dựng Giao diện Web Frontend

Sau khi da co backend tim kiem hoat dong, buoc tiep theo la xay dung giao dien nguoi dung. Frontend se giao tiep voi backend thong qua API endpoint de gui truy van va hien thi ket qua. Ban co the lua chon giua nhieu cong nghe khac nhau tuy theo muc do phuc tap va kinh nghiem cua ban.

7.1. Backend API voi Flask

Flask la Python framework nhe, don gian va phu hop cho viec tao API endpoint cho he thong tim kiem. Ban can tao mot API nhan query string tham so, truy van vao Whoosh index, va tra ve ket qua duoi dang JSON. API nay sau do se duoc goi boi frontend hoac boi WebView trong ung dung APK.

```
from flask import Flask, request, jsonify
from whoosh.index import open_dir
from whoosh.qparser import MultifieldParser
```

```

app = Flask(__name__)

@app.route('/api/search')
def search():
    query_text = request.args.get('q', '')
    page = int(request.args.get('page', 1))
    per_page = int(request.args.get('per_page', 10))

    if not query_text:
        return jsonify({'results': [], 'total': 0})

    ix = open_dir('search_index')
    parser = MultifieldParser(
        ['title', 'content', 'description'], ix.schema)
    query = parser.parse(query_text)

    with ix.searcher() as searcher:
        results = searcher.search_page(
            query, page, pagelen=per_page)
        output = []
        for hit in results:
            output.append({
                'title': hit['title'],
                'url': hit['url'],
                'score': hit.score,
                'snippet': hit.highlights('content',
                    top=3) or hit['description'][:200]
            })
        return jsonify({
            'results': output,
            'total': results.total,
            'page': page,
            'pages': results.pagecount
        })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

7.2. Frontend đơn giản với HTML + JavaScript

Bạn không cần framework phức tạp để tạo giao diện tìm kiếm. Một trang HTML đơn giản với CSS và JavaScript là đủ để bắt đầu. Giao diện bao gồm một ô nhập (input), nút tìm kiếm, và vùng hiển thị kết quả. JavaScript sẽ gửi AJAX request đến backend API và cập nhật giao diện một cách động mỗi khi nhấn Enter hoặc click nút Search.

Để cải thiện trải nghiệm người dùng, hãy thêm các tính năng: tự động hoàn thành (autocomplete/suggestion), highlight từ khóa trong kết quả, phân trang, và

thời gian phản hồi. Bạn cũng nên tối ưu giao diện cho di động (responsive design) vì ứng dụng APK sẽ chạy trên màn hình nhỏ.

7.3. Tựa chọn Framework frontend hiện đại

Nếu bạn muốn giao diện hiện đại và hiệu suất tốt hơn, có thể sử dụng Next.js (React) hoặc Vue.js. Next.js là lựa chọn khuyến nghị vì nó hỗ trợ Server-Side Rendering (SSR), tốt cho SEO và hiệu suất. Bạn có thể tạo một ô tìm kiếm (search bar) với tự động gợi ý (debounced suggestions), hiển thị kết quả với highlight và phân trang, và thêm các tính năng phức tạp như bộ lọc (filter), sắp xếp (sort), và lưu lịch sử tìm kiếm (search history).

Chương 8: Đóng gói APK Android (WebView)

Sau khi có hệ thống tìm kiếm hoạt động (web hoặc backend API), bạn muốn biến nó thành ứng dụng Android để cài đặt trên di động. Phần này sẽ hướng dẫn các cách khác nhau để đóng gói thành APK, từ đơn giản đến nâng cao.

8.1. Phương pháp 1: WebView Activity (Android Studio)

Đây là cách cơ bản nhất để tạo APK từ một trang web. Bạn sử dụng Android Studio để tạo một dự án mới, thêm một WebView Activity, và cấu hình nó để tải trang web của hệ thống tìm kiếm. WebView là một thành phần Android cho phép hiển thị nội dung web bên trong ứng dụng, tương tự như một trình duyệt mini.

Quy trình cụ thể: Đầu tiên, cài đặt Android Studio và tạo một dự án mới với template "Empty Activity". Thứ hai, thêm quyền INTERNET vào file AndroidManifest.xml. Thứ ba, thêm WebView vào layout XML và cấu hình nó trong Activity. Thứ tư, tải JavaScript cho WebView (nếu frontend sử dụng JavaScript). Thứ năm, xử lý các sự kiện như quay lại, tiến tới, và loading. Cuối cùng, build APK và kiểm thử trên thiết bị hoặc emulator.

```
// MainActivity.java
import android.os.Bundle;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.webkit.WebSettings;
import android.webkit.WebChromeClient;
import androidx.appcompat.app.AppCompatActivity;
```

```

public class MainActivity extends AppCompatActivity {
    private WebView webView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        webView = findViewById(R.id.webview);
        WebSettings settings = webView.getSettings();

        // Enable JavaScript
        settings.setJavaScriptEnabled(true);

        // Enable responsive viewport
        settings.setUseWideViewPort(true);
        settings.setLoadWithOverviewMode(true);

        // Configure WebView client
        webView.setWebViewClient(new WebViewClient());
        webView.setWebChromeClient(new WebChromeClient());

        // Load the search engine URL
        webView.loadUrl("https://your-search-app.com");
    }

    @Override
    public void onBackPressed() {
        if (webView.canGoBack()) {
            webView.goBack();
        } else {
            super.onBackPressed();
        }
    }
}

```

8.2. Phương pháp 2: Apache Cordova / Capacitor

Neu bạn quen với web development và không muốn dùng Android Studio, Apache Cordova và Capacitor là những lựa chọn tuyệt vời. Chúng cho phép bao bọc (wrap) một ứng dụng web thành ứng dụng Android chỉ với một vài lệnh đơn giản. Bạn giữ nguyên code frontend (HTML, CSS, JavaScript) và công cụ sẽ tự động tạo project Android, iOS từ đó.

Với Capacitor (khuyến nghị), bạn chỉ cần: thứ nhất, cài đặt Capacitor vào project web của bạn. Thứ hai, chạy "npx cap init" để khởi tạo cấu hình. Thứ ba, chạy "npm run build" để build frontend. Thứ tư, chạy "npx cap add android"

de them platform Android. Thu nam, chay "npx cap open android" de mo project trong Android Studio va build APK. Capacitor cung ho tro truy cap cac API native cua thiet bi nhu camera, storage, va push notification thong qua plugin.

8.3. Phuong phap 3: PWA (Progressive Web App)

PWA la mot cach hien dai de bien trang web thanh ung dung co the cai dat. Ban can them Service Worker va Web App Manifest vao trang web. Service Worker cho phep ung dung hoat dong offline va cache du lieu, trong khi Manifest cung cap thong tin ung dung (ten, icon, mau sac) de trinh duyiet co the cai dat ung dung nhu mot app binh thuong.

De chuyen PWA sang APK (TWA - Trusted Web Activity), ban co the su dung PWABuilder (pwabuilder.com) hoac Bubblewrap. Cong cu nay se tu dong tao file APK tu PWA cua ban, voi uu diem la ung dung se hoat dong nhanh hon vi no su dung Chrome engine cua thiet bi thay vi WebView cu. PWA phu hop nhat khi he thong tim kiem cua ban la mot web app va ban muon nhanh chong tao APK ma khong can viet code native.

Chuong 9: Trien khai va Bao tri He thong

Sau khi xay dung xong, viec trien khai va bao tri he thong la buoc cuoi cung nhung khong ket quan. Mot he thong duoc trien khai dung se hoat dong on dinh, hieu nang cao va de nang cap khi can thiet.

9.1. Trien khai Backend Server

Cho he thong tim kiem can ket noi Internet, ban can trien khai backend len mot server. Lua chon pho bien la su dung VPS (Virtual Private Server) tu cac nha cung cap nhu DigitalOcean, Linode, hoac Hetzner voi gia tu 5 USD/thang. Ban can cai dat Python, Nginx (lam reverse proxy), Gunicorn (WSGI server cho Flask), va cau hinh SSL certificate (Let Encrypt) de bao mat ket noi.

Quy trinh trien khai: Dau tien, thiet lap server voi Ubuntu Server 22.04 LTS. Thu hai, cai dat Python va pip. Thu ba, clone code tu Git repository. Thu tu, cai dat cac dependency (requirements.txt). Thu nam, cau hinh Nginx reverse proxy de chuyen huong traffic den Gunicorn. Thu sau, cau hinh firewall (UFW) de chi mo port 80 va 443. Cuoi cung, dat cron job de chay crawler dinh ky (hang ngay hoac hang tuan) de cap nhat chi muc.

9.2. Trien khai Offline (cho APK)

Neu ban muon ung dung APK hoat dong hoan toan offline (khong can server), ban can dong goi ca database va chi muc vao APK. Cach thuc: Su dung SQLite de luu ca du lieu goc va inverted index. Chuyen SQLite database vao thu muc assets cua project Android. Khi ung dung chay lan dau, copy database tu assets vao thu muc internal storage. Frontend WebView se giao tiep voi backend thong qua localhost (127.0.0.1), khong can ket noi Internet.

Uu diem cua phuong phap nay la ung dung hoat dong bat ky luc nao, khong phu thuoc Internet, va toc do tim kiem cuc ky nhanh vi moi du lieu deu nam tren thiet bi. Nhuoc diem la dung luong APK se lon hon (tuy thuoc vao kich thuoc database), va du lieu khong duoc cap nhat tu dong. Ban can xay dung co che cap nhat thu cong hoac tu dong khi co ket noi Internet.

9.3. Giam sat va bao tri

Cong viec	Tan suat	Mo ta
Chay crawler cap nhat index	Hang ngay/tuan	Thu thap du lieu moi va cap nhat chi muc
Giam sat hieu nang	Lien tuc	Theo doi CPU, RAM, response time, error rate
Sao luu database	Hang ngay	Sao luu tu dong vao cloud storage (S3, GCS)
Nang cap phien ban	Khi can	Update thu vien, fix bug, them tinh nang moi
Kiem tra chat luong index	Hang tuan	Do luong precision, recall cua ket qua tim kiem
Optimize ranking	Hang thang	Phan tich query log, dieu chinh tham so BM25

Bang 4: Lich trinh bao tri he thong khuyen nghi

Chương 10: Tổng kết và Lời khuyên

Xay dung mot he thong tim kiem tu host khong phai la nhieu vu qua kho, nhung no doi hoi su kien thuc ve nhieu linh vuc: web crawling, cau truc du lieu (inverted index), thuat toan (ranking), web development, va mobile development. Bang viec tu xay dung, ban so huu tron ven he thong, khong phu thuoc ben thu

ba, và có thể tùy chỉnh mọi khía cạnh theo nhu cầu cụ thể của mình.

10.1. Lời khuyên cho người mới bắt đầu

Thứ nhất, hãy bắt đầu với phạm vi nhỏ. Không cần xây dựng hệ thống lớn ngay từ đầu. Hãy chọn 5-10 website mục tiêu, xây dựng crawler thu thập khoảng 1000-5000 trang, tạo inverted index với Whoosh, và xây dựng một giao diện đơn giản. Sau khi hệ thống hoạt động tốt, bạn có thể dần dần mở rộng phạm vi. Thứ hai, hãy sử dụng các thư viện có sẵn thay vì tự viết mọi thứ từ đầu. Whoosh, Meilisearch, Scrapy đã được tối ưu hóa và kiểm thử kỹ lưỡng, không cần "reinvent the wheel".

Thứ ba, hãy tập trung vào chất lượng dữ liệu trước khi tối ưu ranking. Một hệ thống với dữ liệu sạch, chỉ mục chính xác và giao diện tốt sẽ mang lại trải nghiệm người dùng xuất sắc hơn một hệ thống với ranking phức tạp nhưng dữ liệu lỗi. Thứ tư, hãy test thử on tất cả các thành phần: crawler, indexer, search engine, frontend và APK. Mọi thành phần đều cần được kiểm thử độc lập trước khi kết hợp chung với nhau. Cuối cùng, hãy đọc thêm về các thuật toán nâng cao như semantic search (sử dụng sentence embeddings), learning to rank, và vector search (FAISS, Milvus) để cải thiện hệ thống trong tương lai.

10.2. Chi phí ước tính

Hang muc	Mien phi	Tra phi (hang thang)
Server VPS	-	5-20 USD (DigitalOcean, Linode)
Domain name	-	10-15 USD/nam
SQLite / Whoosh	Mien phi (tai dia), 0 USD	0 USD
Meilisearch Cloud	Free tier (1 index)	29 USD+ (thang)
CDN / SSL	Let Encrypt (mien phi)	10-20 USD (Cloudflare Pro)
Android Developer	Mien phi (chia se thu cong)	25 USD lan (Google Play)

Bang 5: Chi phí ước tính cho hệ thống tìm kiếm tự host